played - Architecture and Protocols

1. Software Design Pattern: MVVM (Model-View-ViewModel)

played is a game logging and reviewing application where users log games, rate them, write reviews, and follow other users. We use the **MVVM** pattern to keep the user interface separate from the business logic and data access. This makes the app easier to test, maintain, and extend, and lets team members work on screens and data layers in parallel.

2. How the Parts Apply to Our Application

Model

The Model represents the data and the rules of the application. It consists of data classes and a repository layer that fetches and stores data.

- `User` (username, email, profile picture, followers, following)
- `Game` (title, poster, release year, platform, rating, official description)
- `Log` (user, game, played/liked/wishlist status, star rating)
- `Review` (user, game, text, rating)
- Repositories: `AuthRepository`, `GameRepository`, `LogRepository`, `UserRepository`, which hide whether data comes from the server or a local cache.

View

The View is the UI designed in Figma and implemented as screens. It only displays data and forwards user actions to the ViewModel.

- **Welcome Screen:** shows the "played" title and navigates to Login after a short delay.
- **Login / Sign Up Screens:** input fields for username, email, password, and confirm password, a terms and conditions checkbox, and a submit button.
- **Home Screen:** horizontally scrollable *Trending Games* and *Reviews from Friends*.
- **Search Screen:** search bar and *Recently Searched* list.
- **Game Detail Screen:** poster, release year, platform, score, official description, Log button, and scrollable user reviews.
- **Log Screen:** poster info, Played / Like / Wishlist buttons, 5-star rating, and Add Review field.

- **Profile Screen:** profile picture, username, game and review counts, *Favorite Games* , *Played Recently*, and buttons to the Games, Followers, Following, and Reviews screens.
- **Games, Followers, Following, and Reviews Screens:** lists of games (Played, Wishlist, Likes), mutual follows, and the user's reviews with poster and rating.
- **Bottom navigation bar (HUD):** Home, Search, and Profile tabs, with the icon of the active page highlighted.

ViewModel

The ViewModel holds the state of each screen and exposes it to the View, which observes it and updates automatically. It calls the repositories and contains no UI code.

- `AuthViewModel` : validates input (empty fields, password match, terms accepted) and handles login and sign up.
- `HomeViewModel` : loads trending games and friends' reviews.
- `SearchViewModel` : queries games and manages the recent search history.
- `GameDetailViewModel` : loads game information and user reviews.
- `LogViewModel` : saves played/like/wishlist status, the star rating, and the review.
- `ProfileViewModel` : loads user statistics, favorite games, recent games, and follow lists.

Data Flow Example

The user opens a game and presses Log. The Log Screen (View) sends the action to `LogViewModel` , which asks `LogRepository` to save the log. The repository sends the request to the backend, and the result updates the ViewModel state, so the Log Screen and the Profile counters refresh without any manual UI handling.

3. Key Protocols and Interfaces

Protocol / Interface	Purpose in the app
HTTPS	Encrypted transport for all communication between the app and the server, protecting passwords and user data.

Protocol / Interface	Purpose in the app
REST API (JSON)	Main interface between the app and the backend: login, sign up, fetching trending games, saving logs and reviews, follow/unfollow, and profile data. Data is exchanged as JSON.
Token-based authentication (JWT)	After login or sign up, the server returns a token that is attached to each request, so the user stays signed in and the server can identify who is logging or reviewing.
External game database API (e.g., RAWG or IGDB)	Provides game information such as posters, release years, platforms, scores, descriptions, and search results.
Local storage / secure storage	Stores the auth token securely and caches recent searches and recently viewed data for faster loading and basic offline use.
Image loading over HTTPS (with caching)	Loads and caches game posters and profile pictures so scrolling lists stay smooth.
Navigation interface	Handles screen transitions (Welcome → Login → Home, bottom bar tabs, Game Detail → Log, Profile → sub-pages) and keeps the HUD state consistent.
Repository interfaces	Abstract data sources so ViewModels do not depend on a specific API, which allows mock data during development and testing.